Portland State UNIVERSITY

PROJECT PROPOSAL

Formal Property Verification and CDC Analysis of an Asynchronous FIFO

Applying a golden-validated formal flow to a design that already passed UVM simulation

Author	Aravindh Nanjaiya Latha
Program	M.S. Electrical and Computer Engineering, Portland State University
Course	ECE 506 — Project (individual)
Faculty advisor	Dr. Xiaoyu Song
Project coordinator	Pratibha Munnangi
Design stage	Functional verification and CDC sign-off
Repository	`github.com/aravindh-nanjaiya-latha/async_fifo_fv`
Submitted	November 2026

Contents

1 Introduction 3
2 Designs Under Verification 3
2.1 ECE 593 Asynchronous FIFO 3
2.2 Golden Reference FIFO 3
3 Project Goal 4
4 Methodology 5
4.1 Four-Stage Flow 5
4.2 Verification Scope 5
5 Preliminary Baseline 6
6 Expected Trade-offs 7
7 Tools and Environment 7
8 Timeline, Deliverables and Risks 7
9 Attribution and Academic Integrity 8
10 References 8

Figures

Figure 1 FIFO architecture and verification hooks 4
Figure 2 Four-stage verification methodology 5
Figure 3 Verification scope 6

Tables

Table 1 Preliminary baseline: golden reference versus ECE 593 FIFO 7
Table 2 Timeline and milestones 8

1. Introduction

In ECE 593 (Pre-Silicon Validation) my team and I designed an asynchronous FIFO and verified it with a UVM testbench. The design passed its simulation regression and was submitted. This project asks the question that simulation cannot answer: is the design actually correct? Formal property verification (FPV) and formal clock-domain-crossing (CDC) analysis are applied to the same RTL, unmodified, to measure what a formal flow finds that the UVM environment did not. The defects found are then fixed with minimal changes, and the identical flow is re-run so that the improvement is measured under the same tools and settings.

The chip-design stage addressed is functional verification and CDC sign-off. The FIFO under verification is declared reuse of my own past work (Section 9); the formal checkers, CDC methodology, baseline measurement, fixes and regression are new work for ECE 506.

2. Designs Under Verification

2.1 ECE 593 Asynchronous FIFO

The design under verification has two independent clock domains; 6-bit binary pointers over the range 9..54 (46 memory slots, 45 usable entries, because `full` is detected as `b_wptr_next == b_rptr_sync`); gray-coded pointers that cross domains through two-flop synchronisers; combinational flag logic; and the original UVM testbench. The RTL is frozen under `rtl/` at git tag `baseline-renamed`. Its non-power-of-two depth and offset pointer range make it a more demanding target than a textbook FIFO.

2.2 Golden Reference FIFO

A depth-16 textbook gray-pointer asynchronous FIFO, generated with Claude and declared as such, serves as the reference. It is not the subject of the project. Its role is to validate the verification infrastructure: a checker that fires on a correct design is a broken checker, so every property and every CDC configuration is proven on the golden FIFO before it is trusted on the ECE 593 design.

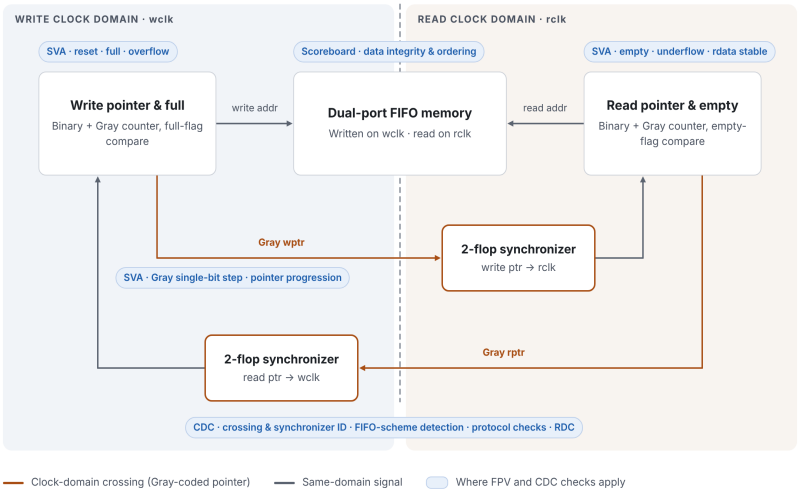


Figure 1. FIFO architecture and where the FPV and CDC checks attach. The golden reference is shown; the ECE 593 FIFO has the same structure with 6-bit pointers over 9..54.

3. Project Goal

Measurable target. Take the ECE 593 FIFO from 11 of 20 formal assertions proven to 20 of 20, and its CDC report from unmeasured to zero errors, using the same checker and CDC script that already pass on the golden FIFO (22 of 22 assertions proven, zero CDC errors). RTL changes are confined to the lines that cause the failures, verified by `git diff baseline-renamed -- rtl/` showing only fixes.

Reported alongside. Counterexample count and trace lengths, cover bounds (for example, cycles to first `full`), CDC pair and violation counts, lines of RTL changed, and formal proof time.

4. Methodology

4.1 Four-Stage Flow

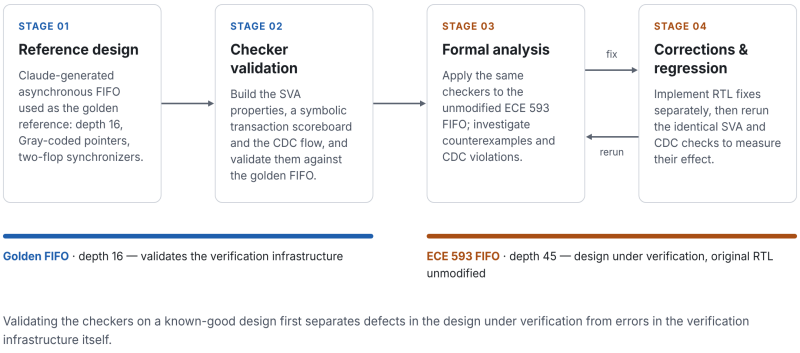


Figure 2. Four-stage verification methodology: the checkers are proven on a known-good reference before they are trusted on the design under verification.

1. **Reference design.** The golden FIFO serves as the known-good control.
2. **Checker validation.** One bound-in SystemVerilog checker (assertions and covers only) covers reset behaviour; overflow and underflow protection; read-data stability; a symbolic transaction scoreboard, in which one undriven tag is treated by the engine as an arbitrary constant so that a single property covers data integrity, ordering, no-drop and no-duplicate for every transfer; pointer step and hold; gray-code single-bit change, including at the wrap, and encoding; occupancy-versus-flag checks built from port events only; and bounded response, requiring a flag to follow true occupancy within the synchroniser latency. One CDC script configures asynchronous clock mode, extracts crossings and synchronisers, detects the FIFO scheme, runs functional protocol checks and reset-domain analysis. Both are proven clean on the golden FIFO.
3. **Formal analysis of the ECE 593 design with the RTL untouched.** Expected results are written down before each run; the preliminary run already overturned two of my own predictions (Section 5). Every counterexample is triaged as design defect versus checker error before it is called a bug. In CDC, the design's own push/pop guards are proven as custom protocol checks rather than assumed away — an assumption on `!(r_en && empty)` would have hidden Bug 3 — and the gray-code declaration is deliberately not used, because the tool believes declarations and would auto-waive the very violation it should report.
4. **Corrections and regression.** One commit per defect with a minimal diff, after which the identical FPV and CDC scripts are re-run. A counterexample on `baseline-renamed` paired with a proof on the fix commit is the before/after evidence.

4.2 Verification Scope

Figure 3 summarises the property groups and CDC phases applied, unchanged, to both designs.

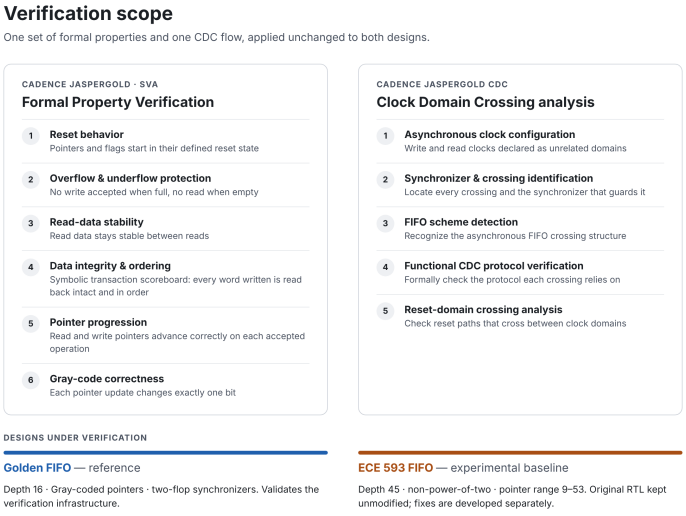


Figure 3. Verification scope: one set of formal properties and one CDC flow, applied to both designs.

5. Preliminary Baseline

The flow was run on both designs on 29 October 2026 with Cadence Jasper 2026.06 on the PSU CECS server deimos. The golden column is the control; the ECE 593 column is the starting point the project improves on.

Measurement	Golden FIFO (control)	ECE 593 FIFO (baseline)	Interpretation
FPV assertions proven	22 / 22, full proofs	11 / 20	9 counterexamples: 7 from design defects, 2 from checker calibration (pointer range 9..54 rather than the assumed 9..53), corrected before the final baseline run
Data integrity and ordering	proven	CEX, 227 cycles	wrong or lost data is reachable
Empty asserted at zero occupancy	proven	CEX, 2 cycles	Bug 1: read-side synchroniser resets to 0, not gray(9); the FIFO reads while empty immediately after reset and never reports empty again
Read pointer holds without a read	proven	CEX, 51 cycles	Bug 3: pointer wrap 54→9 is not gated by the read enable; slot 54 is skipped unless read in that exact cycle
Gray pointer changes one bit, incl. wrap	proven	proven	suspected defect refuted: 54 = ~9, so the wrap is gray-safe by construction
Write pointer holds without a write	proven	proven	suspected mirror of Bug 3 refuted
CDC (Jasper CDC app)	18 pairs, 1 FIFO scheme, 0 errors	not yet run	scheduled for the week of 2 November
ECE 593 UVM regression	—	passed	none of the above was detected in simulation

Table 1. Preliminary baseline: golden reference versus ECE 593 FIFO.

The two refuted suspicions are kept in the bug report as retractions. A bug list with no retractions is not a credible bug list, and they are the evidence that the checker is not simply confirming what was expected.

6. Expected Trade-offs

- **Added logic.** An extra mux term in each pointer's next-state logic and a parameterised synchroniser reset value — small, but measured in lines changed and, where practical, a synthesis gate-count estimate.
- **Verification cost.** Formal proof time against UVM regression time; the scoreboard counter width (SEQW) required for soundness; bounded-response bounds in place of liveness.

- **Obligations pushed outward.** Reset release is assumed synchronised upstream and recorded as an explicit assumption for the integrator rather than hidden in a clean report.

7. Tools and Environment

- Cadence JasperGold 2026.06 — FPV and CDC apps, on PSU CECS `deimos` / `phobos`
- SystemVerilog-2012 and SystemVerilog Assertions
- Tcl run scripts carrying an expected-result header, so that a re-run is a diff rather than a re-interpretation
- Git and GitHub: tags `baseline-upstream`, `baseline-renamed`, `fv-infra`; one commit per fix; collaborators songx@pdx.edu and pratibha@pdx.edu

8. Timeline, Deliverables and Risks

Week	Work	Milestone
2 – 6 Nov	Final FPV baseline with corrected checker constants; CDC baseline of the ECE 593 FIFO; proposal	Proposal, 6 Nov
7 – 13 Nov	Fix Bug 1 (and its write-side twin) and Bug 3, one commit each; re-prove; triage the data-integrity counterexample	FPV after-column
14 – 20 Nov	CDC on the fixed RTL to zero errors; any residual finding fixed or documented with evidence	Before/after table
21 – 27 Nov	Report draft; README how-to-run verified from a clean clone	Report draft
28 Nov – 2 Dec	Demo video (≤30 min: problem, approach, live Jasper run, a challenge, results, looking back); final submission	Submit, 2 Dec
3 – 9 Dec	Buffer	—

Table 2. Timeline and milestones.

Deliverables. GitHub repository with RTL, checkers, scripts, run logs, bug report and a README with how-to-run; a 5–10 page final report with the before/after table and trade-off analysis; a demo video on PSU Google Drive shared with the advisor and coordinator.

Risks. Jasper is available only on deimos/phobos. CDC FIFO-scheme detection may suppress a convergence check on the ECE 593 design; a diagnostic run with detection disabled is planned. The data-integrity counterexample may reveal a third defect beyond the two confirmed.

9. Attribution and Academic Integrity

- **ECE 593 FIFO RTL and UVM testbench:** developed collaboratively by Aravindh Nanjaiya Latha, Maithreyi Venkatesan and Raghavendra Davarapalli (original repository `RaghaHanuma22/Async_FIFO`). Reused here as the design under verification, unmodified at baseline; declared reuse of my own past work.

- **Golden FIFO:** generated with Claude (Anthropic) and declared as such in the file header and README; used only as a reference to validate the checker, and independently evaluated by the same formal flow.
- **Formal checkers, CDC methodology, scripts, analysis, fixes and report:** my work for ECE 506. An AI assistant was used for review and challenge questions; every property and configuration in the repository was written, debugged and can be explained by me, and the git history shows the work in sequence.

10. References

[1] C. E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design,” SNUG San Jose, 2002.

[2] Cadence Design Systems, JasperGold Formal Verification Platform and JasperGold CDC App documentation, release 2026.06 (licensed; not redistributed).

[3] A. Nanjaiya Latha, M. Venkatesan, R. Davarapalli, Async_FIFO, ECE 593 project repository, `github.com/RaghaHanuma22/Async_FIFO`.

[4] IEEE Std 1800-2017, IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language.